Predicting Clothing Popularity

Using Machine Learning Techniques

Hannah Hedima

ITCS-3156

Professor Minwoo Lee

May 2026

1: Introduction

Fashion is a rapidly evolving industry, and for companies, understanding consumers' interests is very important for a company's success. Understanding what pieces of clothing are considered popular can help businesses make smarter decisions that improve sales. In this project, I will use machine learning classification models that will predict a clothing item's popularity based on customer review data from the Women's Clothing E-Commerce Reviews dataset from Kaggle. In this project, I aim to compare two different machine learning models: a linear regression model using gradient descent, and a perceptron classifier that classifies clothing items as popular or not popular.

2: Data

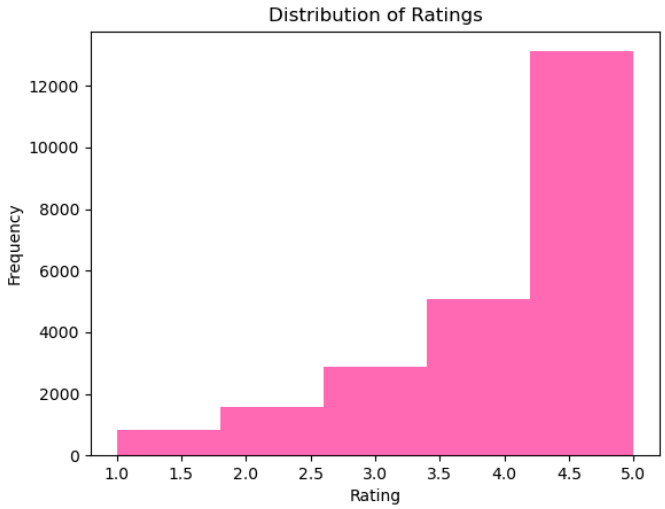
2.1: Dataset

The dataset used in this project has customer reviews of women's clothing items. There are 23,486 entries and 11 features. Each row is a review that has numerical and categorical information. The features included are the age of the reviewer, rating of the product, if the customer recommends the product, the positive feedback count, and the department name and class name to describe the type of clothing.

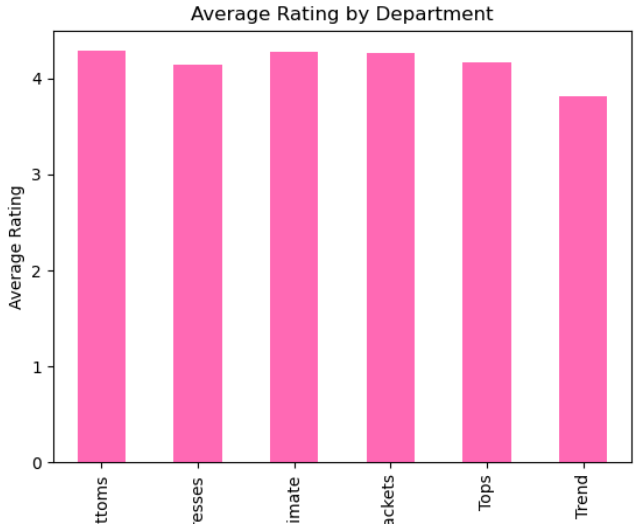
To make this data work with classification, I created a new variable called "popular." If the item had a rating of 4 or higher, it was labeled as popular, which is equal to 1, while those that were below 4 were labeled as not popular, which is equal to 0. This dataset is unbalanced, so there happened to be more popular items than non-popular items. This is something to keep in mind when analyzing how the model learns and viewing its results.

2.2: Visual Analysis

To get a better understanding of the dataset, I made several visualizations to explore the relationships between each feature and item popularity.



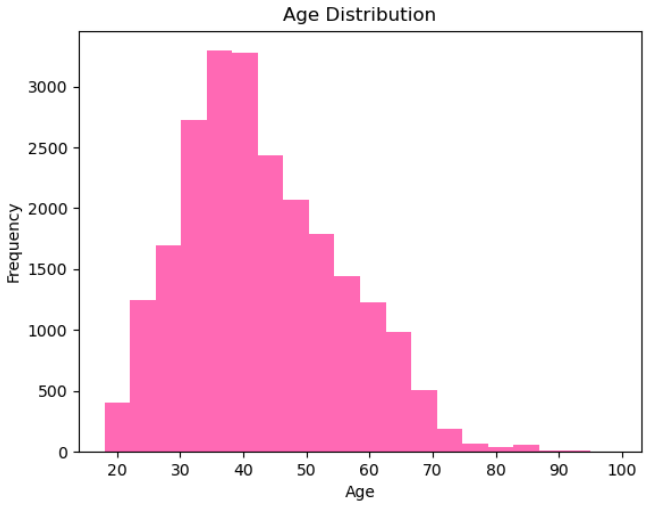
This histogram shows item ratings; most items received high ratings, between 4 and 5. Showing what I mentioned earlier about the data being unbalanced, with most items being "popular."



This bar chart compares the average ratings across various clothing categories. The categories shown in this chart(left to right) are bottoms, dresses, intimates, jackets, and tops. This chart tells

us that these items typically get high ratings, with bottoms getting the highest. This could also tell us that a certain item type might influence a customer's satisfaction and popularity.

This scatter plot compares positive feedback count versus rating, and this shows that items with higher ratings usually receives more positive feedback. But the outliers in this plot give us a hint that other factors can contribute to the popularity of a piece of clothing.



Finally, in this distribution plot, it shows the reviewers' ages, and most reviewers are about middle age (30- 40 years old). The age of the reviewers can help us understand a slight bias in

the preferences for an item of clothing. These visualizations show us the patterns in the data and gives a good view of the features that could be important in predicting clothing item popularity.

2.3: Preprocessing

Before implementing the machine learning models, I needed to clean up the data a bit and make a few preprocessing steps. First, I handled missing values by removing rows with missing values in key columns, and I filled these missing values with empty strings.

```
15    df = df.dropna(subset = ["Rating", "Department Name", "Class Name"])
16    df["Review Text"] = df["Review Text"].fillna("")
```

I made relevant numerical and categorical features that were going to be used for model training; the features used were age, positive feedback count, department name, and class name. Then I categorized variables, and variables like department and class names were converted into numerical form using one-hot encoding. Here, I also converted the ratings into a binary rating and created a binary target based on item ratings. This was essential because my models could not process the text categories.

```
31    #encoding data
32    X = pd.get_dummies(X, columns=["Department Name", "Class Name"])
```

Finally, I split the data using a random seed of 42 into training and testing sets, then normalized numerical features using standard scaling. This is very important for gradient descent and perceptron models, as it makes sure that all features are being used during training.

```
34    #normalizing and training
35    scaler = StandardScaler()
36    X_scaled = scaler.fit_transform(X)
37    X_train, X_test, y_train, y_test = train_test_split(
38        X_scaled, y, test_size=0.2, random_state=42
39    )
```

3: Methods

3.1 Gradient Descent

The first model that I used is a linear regression model trained using gradient descent. Gradient descent is an optimization algorithm that minimizes a loss function by updating model parameters in the direction of the negative gradient (GeeksforGeeks). This allows the model to gradually reduce the prediction error over multiple iterations. The prediction is computed as a linear combination of input features, where w is the weight vector, x is the input feature vector, and b is the bias term:

$$\hat{y} = w^T x + b$$

To measure the prediction error, the mean squared error loss function is used:

$$J(w) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Gradient descent minimizes loss by iteratively updating the model parameters in the direction that reduces error, where α is the learning rate controlling step size:

$$w := w - \alpha \frac{\partial J}{\partial w}$$

During training, the model goes through many epochs, and the loss is recorded at each iteration to monitor convergence. The decreasing loss with each iteration shows that the model is successfully learning the patterns in the data. Even though linear regression creates continuous outputs, the final predictions were converted into binary classes that used a threshold of 0.5. Values that were greater than or equal to 0.5 were classified as popular, while those below 0.5 were classified as not popular.

3.2: The Perceptron

The second model used is the perceptron, it is a linear binary classifier. Unlike with gradient descent that minimizes a continuous loss function, the perceptron focuses on classification correctness. The decision rule for the perceptron is:

$$\hat{y} = \begin{cases} 1 & \text{if } w^T x + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

The perceptron updates the weights only when a misclassification happens, the rule update is:

$$w := w + \alpha(y - \hat{y})x$$

So when the model predicts incorrectly, the weights are adjusted in a direction that reduces future errors. During the training process, there are multiple epochs, at each one, the number of misclassifications is recorded. Over time, the model corrects itself and reduces classification errors by changing the decision boundary. The perceptron assumes that the data is linearly separable; this assumption is not always right, which can limit the model's ability to get perfect accuracy.

4: Results

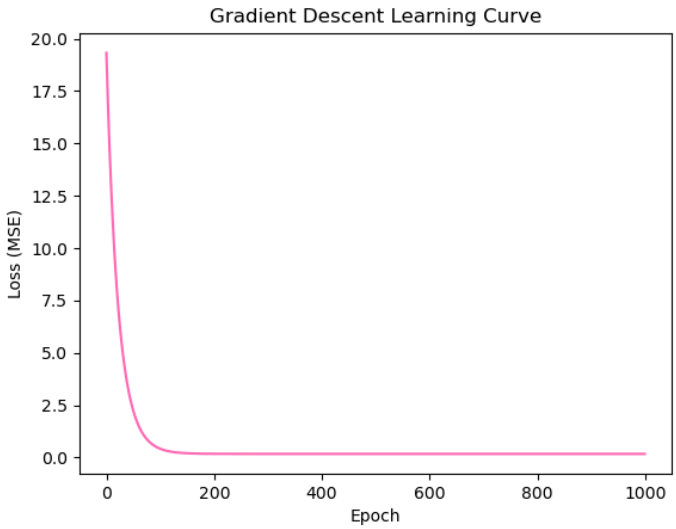
4.1: Experiment Setup

To solve the problem of predicting clothes item popularity, the gradient descent linear regression model and the perceptron classifier were trained using the same dataset and preprocessing pipeline. The dataset was split into an 80% training data and a 20% testing data using a random seed of 42 to make sure it is reproducible. Both of the models were trained using the same feature set, which had numerical features and encoded categorical variables. Feature scaling was added before training to make sure that there is a consistent input range. The gradient descent

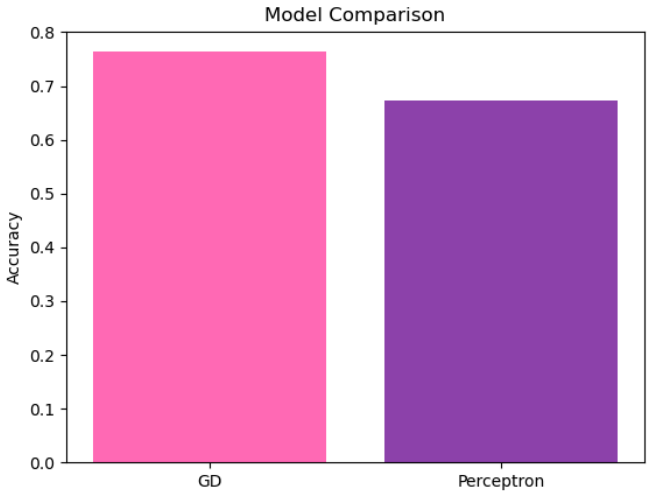
model was trained for 1000 epochs, and the loss was recorded at each epoch. The perceptron model was trained for 100 epochs, and the misclassifications were also recorded at each epoch.

4.2: Model Performance

During training, the gradient descent model was consistently improving, the loss rapidly decreased in the beginning and gradually stabilized, showing that the model successfully converged. The graph below it shows the loss curve and how the model effectively learned the patterns and reached a solution.



When evaluated on the test set, the gradient descent model had an accuracy of about 0.76, showing a strong performance for the classification task. But the perceptron model showed a different pattern during training. The number of misclassifications decreased slowly at the beginning, but it quickly stabilized and showed little improvement in the later epochs. This could lead us to conclude that the perceptron struggled to learn the structure of the data. The accuracy of the perceptron model was about 0.67, which is lower than that of the gradient descent model. The bar chart below clearly shows that the gradient descent performed better than the perceptron.



4.3: Analysis

The results tell us that the gradient descent model is more effective in predicting clothing item popularity than the perceptron. Gradient descent minimizes a continuous loss function which allows it to make gradual and more precise adjustments to the model parameters. On the other hand, the perceptron updates the weights only when there are misclassifications and assumes that the data is linearly separable. Because this dataset has real-world customer review data, the relationship between the features and popularity is more complex and not as perfectly separable. Also, the dataset is imbalanced, so this probability influenced both models, leading them to favor picking the popular (the majority class). In the end, though, the gradient descent model showed the better convergence behavior and higher accuracy, making it the best-suited model out of the two for this problem.

5: Conclusion

5.1: Closure

In this project, two machine learning models were implemented and compared to predict the popularity of clothing items using customer review data. The models used and compared were a linear regression model trained using gradient descent and a perceptron classifier. The results showed that the gradient descent model had higher accuracy and had more stable convergence during training. But the perceptron model showed limited improvements and stabilized early, struggling to learn from the dataset. This shows how model assumptions can impact performance on real-world data. This project showed that optimization-based models like gradient descent are best suited for these types of problems compared to simpler ones, such as classification models like the perceptron.

5.2: Challenges

A major challenge I had while doing this project was working with a real-world dataset, and it missing values and was unbalanced. Handling the missing data was a little bit tedious. The imbalanced dataset, it made it a little hard for the models to learn meaningful patterns. Also, with the topic I chose, typically, people who make reviews on clothing items are biased, so that makes it a little harder to find more meaningful data.

5.3: Final words

There are many ways that this project could be improved. One improvement that could be made was using more advanced machine learning models like K-nearest neighbors, which could probably get more complex relationships in the data. Another thing that could be improved is to address the class imbalance by resampling or weighting, which could help improve model performance on the minority classes. Another thing that is a result of my limited knowledge is using natural language processing techniques to analyze review text, which could give more insight and knowledge on a customer's opinion and improve prediction accuracy. Lastly, further

tuning of hyperparameters and experimenting with different feature combinations could lead to better performance. This project strengthened my understanding of how different machine learning algorithms can learn from data.

6: References/Acknowledgements

6.1: Dataset

Dataset: "Women's Clothing E-Commerce Reviews." Kaggle, https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews.

6.2: References

"Gradient Descent in Machine Learning." GeeksforGeeks, https://www.geeksforgeeks.org/gradient-descent-in-machine-learning/.

"Linear Regression in Machine Learning." GeeksforGeeks, https://www.geeksforgeeks.org/ml-linear-regression/.

Lee, Minwoo, and Hongfei Xue. "The Perceptron Algorithms." ITCS 3156: Introduction to Machine Learning, University of North Carolina at Charlotte. Course Lecture.

"matplotlib.axes — Matplotlib 3.10.8 Documentation." Matplotlib Documentation, https://matplotlib.org/stable/api/axes_api.html.

6.3: Acknowledgments

Code used in this project is from previous homework assignments, Module 5: Gradient Descent and Least Mean Square and Module 7: Perceptrons. AI was used to help with the grammar of the paper. No other resources in this project besides the ones listed were used in this project.

7: Source Code

https://github.com/hammah127/itcs3156-final-project